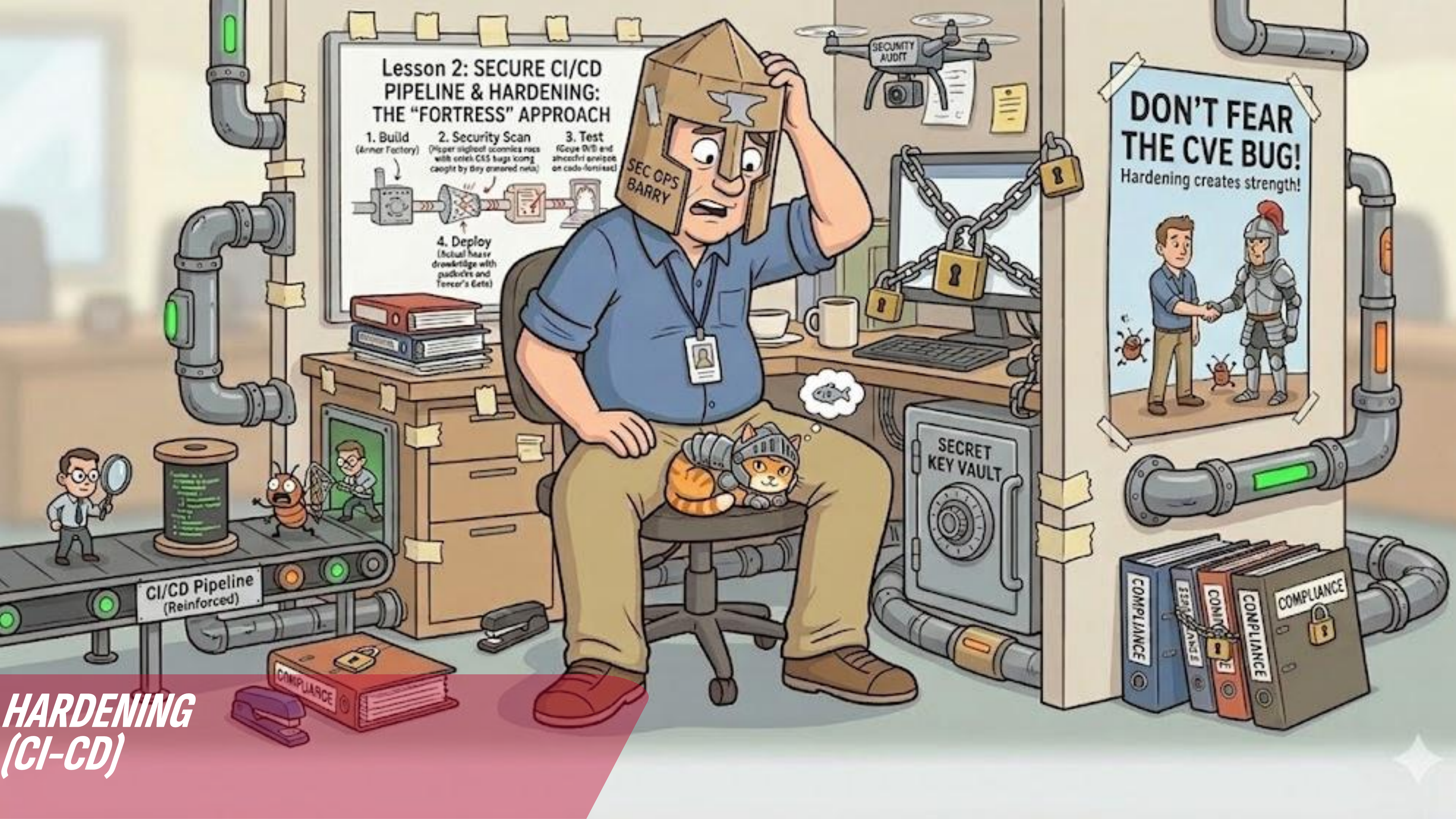




**HARDENING
(CI-CD)**

WAAR STAAN WE

Workshop	Thema	Deliverable
WS01	Wet- & regelgeving, NEN-7510, audit mindset	Begrippenkader
WS02	Hardening CI/CD pipeline (CodeQL, Dependabot, SBOM)	Beveiligde pipeline
WS03	Healthcare Risk Assessment (risicomatrix, bow-tie)	Risicorapport
WS04	Compliance Scanning (CVE/CVSS, Snyk, security backlog)	Scanrapport + backlog
WS05	Secure Coding & Privacy by Design	Logging + tests
WS06	Formeel auditrapport	Definitief auditrapport

AGENDA

- ✓ Terugblik workshop 1
- ✓ Waarom Secure CI/CD (in de zorg)
- ✓ Van DevOps naar DevSecOps
- ✓ Shift left – Security in elke stap
- ✓ NEN-7510:2024-2 relevante controls
- ✓ Deep Dive – Hardening per control
- ✓ Tooling
- ✓ OTAP inrichten
- ✓ Opdrachten

LEERDOELEN

Na deze workshop kun je...

- ✓ Uitleggen waarom een geharde CI/CD-pipeline verplicht is in een healthcare-context
- ✓ De relevante NEN-7510:2024-2 controls koppelen aan concrete pipeline-maatregelen
- ✓ Een GitHub-repository inrichten met branch-protection, SAST, SCA en SBOM
- ✓ OTAP-omgevingen opzetten met gescheiden secrets en deployment-rules
- ✓ Een compliance-verslag schrijven dat aantoont hoe de pipeline voldoet aan NEN-7510

Terugblik

- Wet vs. verordening vs. richtlijn vs. norm
- NEN-7510:2024 = NL-zorgspecifieke uitwerking van ISO 27001/2
 - Deel 1 → organisatie (ISMS)
 - Deel 2 → technische & organisatorische beheersmaatregelen
- Audit mindset: "Niet gelogd = niet gebeurd"
- Compliant by design: regelgeving in requirements, niet een afvinklijstje achteraf

Waarom Secure CI-CD?

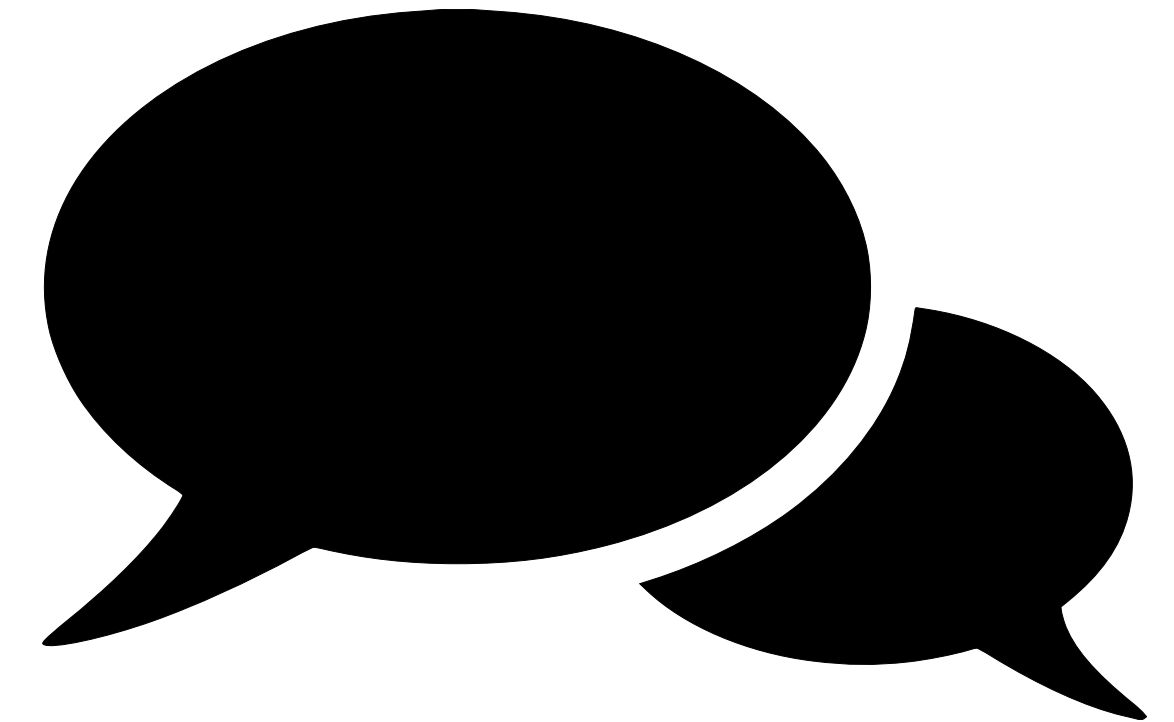
Patiëntveiligheid en compliance

- Medische gegevens zijn bijzondere persoonsgegevens (strengste AVG-categorie)
- Een fout in productie kan direct patiëntveiligheid raken
- IGJ (Inspectie Gezondheidszorg en Jeugd) eist aantoonbaar werken
 - O.a. conform NEN-7510
- Bij een incident moet je kunnen aantonen:
 - Wie heeft deze wijziging goedgekeurd?
 - Welke tests zijn uitgevoerd vóór de release?
 - Wanneer is de software in productie gegaan?
- Wetgeving die dit afdwingt:
 - NIS2 → CBW, CRA, AVG/GDPR
- Insider threats en menselijke fouten zijn de meest voorkomende oorzaak van datalekken

Waarom Secure CI-CD?

“Insider threats en menselijke fouten zijn de meest voorkomende oorzaak van datalekken”

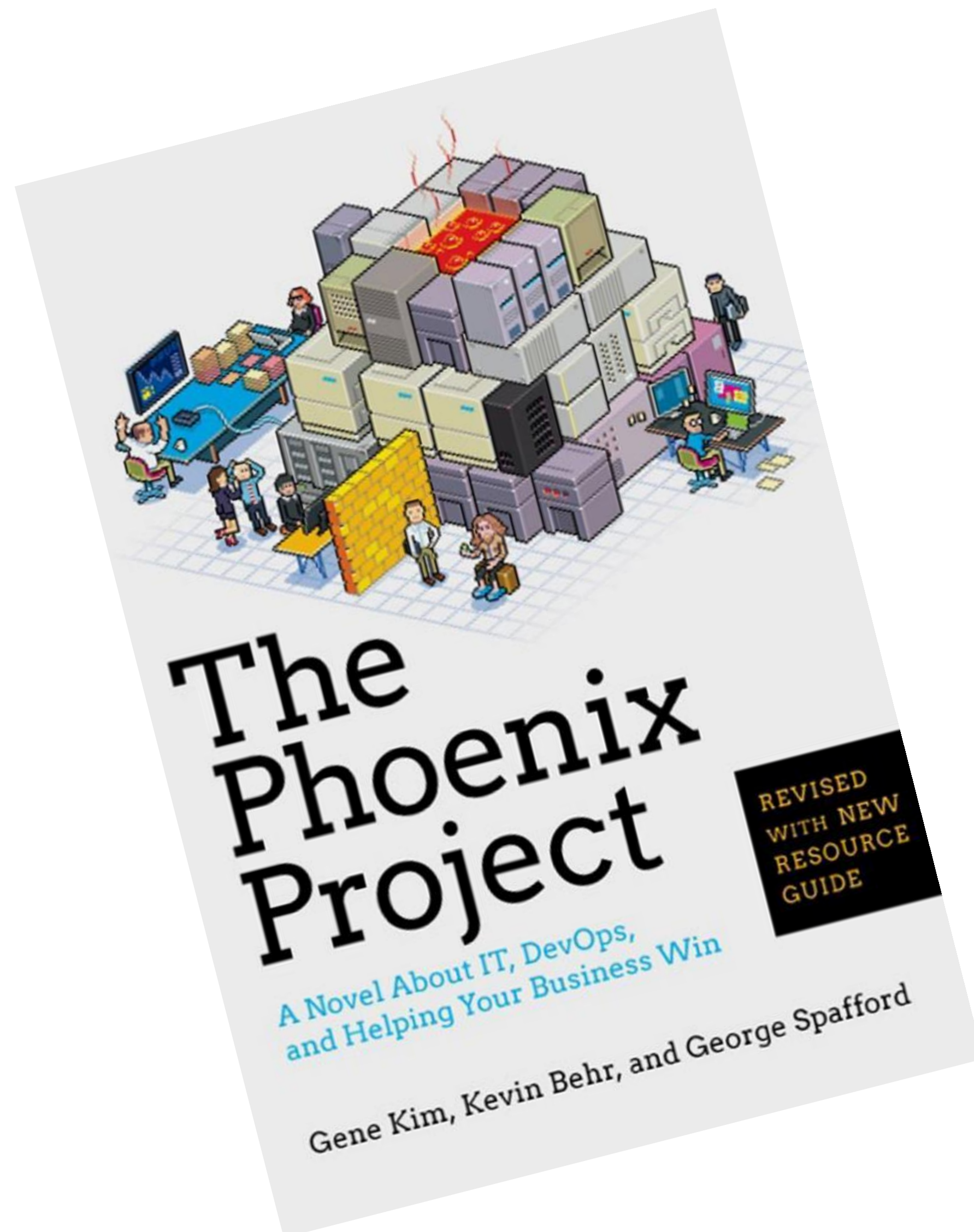
- Wat betekend dit?
- Is dat zo? Voorbeelden?



DevOps?

- Vroegah: Dev bouwt software → gooit het "over de muur" → Ops draait het → problemen zijn elkaars schuld.
- DevOps mixt development en operations en automatiseert zoveel mogelijk
 - "Make the right way the easy way"
- Automatisering staat centraal
 - bouwen, testen en uitrollen ***zonder** handmatige stappen
- Snelle feedback loops
 - een fout wordt binnen minuten zichtbaar, niet pas bij een release over zes weken
- Infrastructure as Code (IaC)
 - servers en omgevingen worden beheerd als code — herhaalbaar en onder versiebeheer
- Geen 'tool' maar cultuur en werkwijze
 - De pipeline is een technische middel om de werkwijze af te dwingen

DevOps | Leestip – The Phoenix Project



- Fictief verhaal maar beschrijft waarom DevOps 'een ding' is geworden.
- Qua invloed vergelijkbaar met 'Clean Code'

DevSecOps?

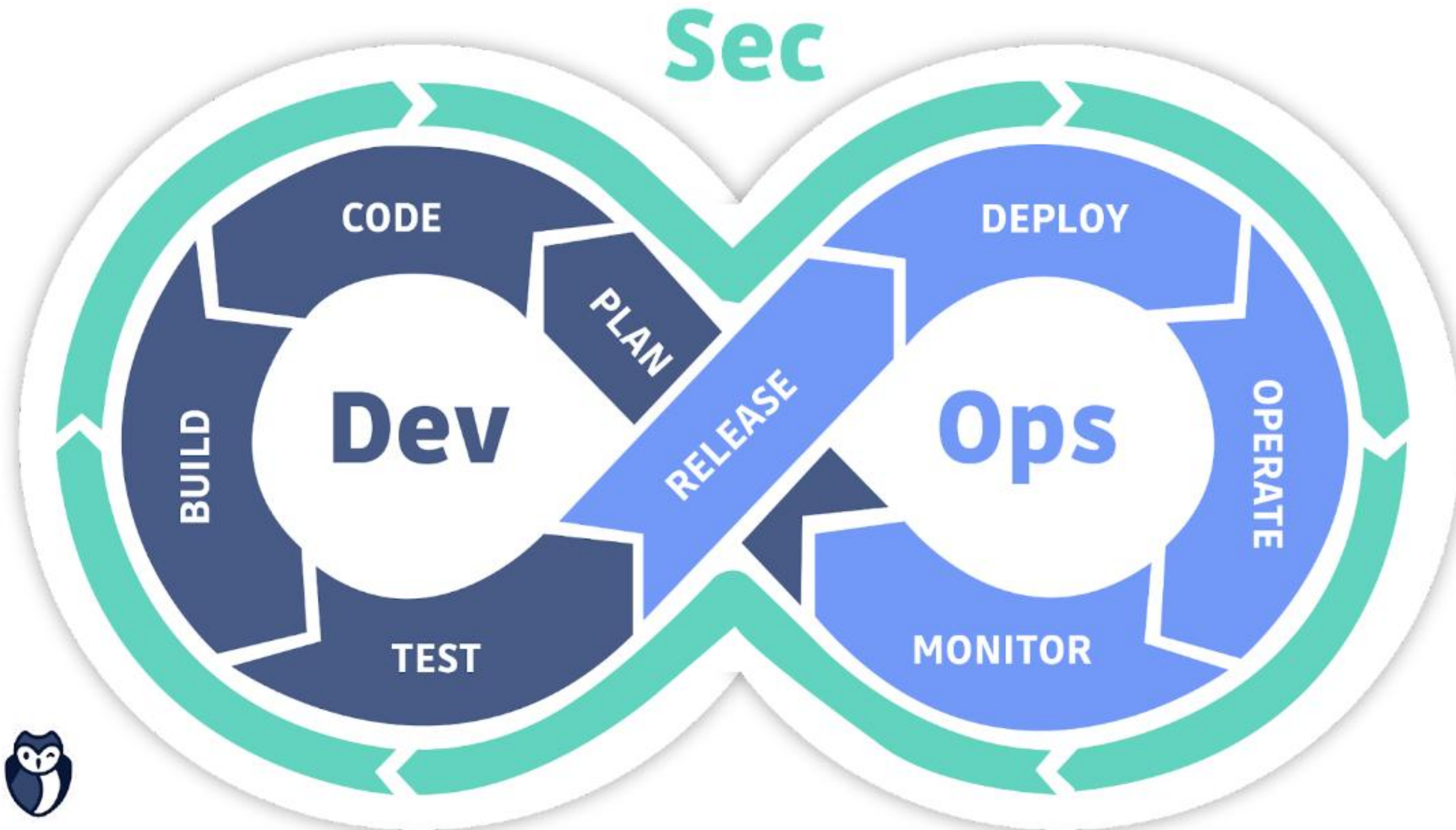
Evolutie na de 'revolutie' van devops

Fase	Verandering
DevOps	Dev + Ops samenwerken; CI/CD automatiseert build & deploy
+ Quality Gates	Geautomatiseerde checks voorkomen 'oepsies' in productie
DevSecOps	Security wordt een gedeelde verantwoordelijkheid van het hele team ('shift left')
Meetbaar & aantoonbaar	Logs, alerts, reviews en pipeline-rapportages = bewijs

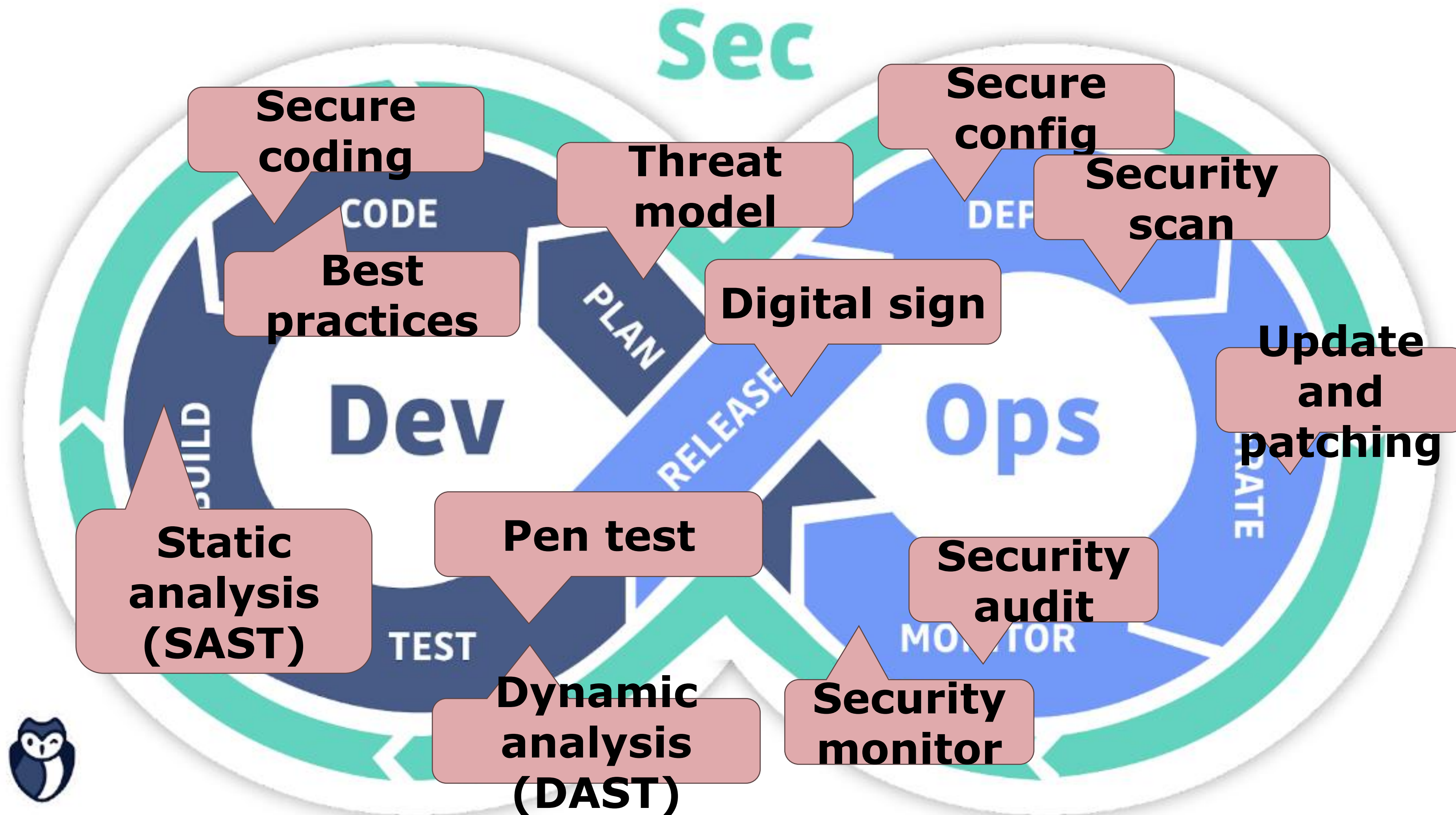
Security 'shift left'

Beveiliging wordt zo vroeg mogelijk in de SDLC ingebracht — niet pas bij de pentest.

DevSecOps: 'Security is a team effort'



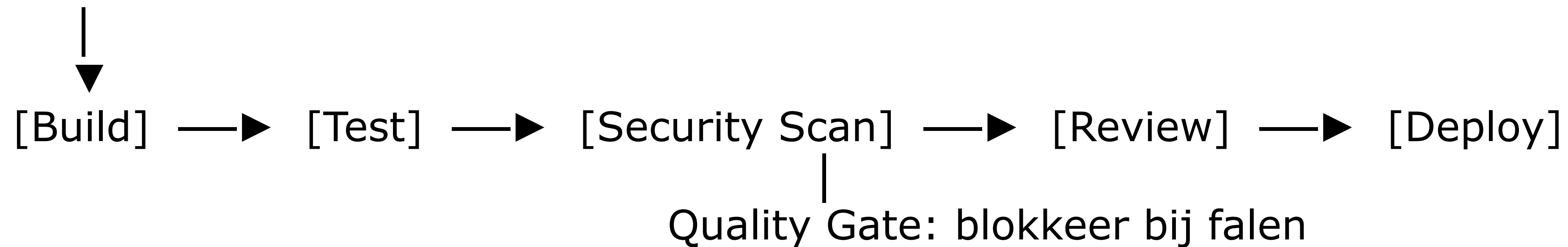
DevSecOps: 'Make the good path the easy path'



CI-CD pipeline?

- De 'bouwstraat' (ontwikkelstraat)

Code commit



- Continuous Integration (CI)
 - elke commit triggert automatisch build + tests
- Continuous Delivery/Deployment (CD)
 - Alleen gevalideerde code gaat door naar (acceptatie/)productie
- Quality gates
 - Checkpoints die de pipeline stoppen als iets niet klopt
- In een secure pipeline is elke stap bewijs (audit trail)

DevSecOps | 'Security in elke fase'

Geen 'discrete' stap, het zit overal

SDLC fase	Security act.	Tooling
Plan / Design	Threat modeling	STRIDE, OWASP Threat Dragon, Iridius Risk*
Code	Secure coding standards, secrets-beheer	Linters, git-hooks
Build	SAST, dependency check	CodeQL, Snyk, OWASP Dependency-Check
Test	DAST, integratie-security tests	OWASP ZAP
Release	SBOM genereren, artifact signing	CycloneDX, Sigstore/Cosign
Deploy	Omgevingsscheiding, least privilege	GitHub Environments
Operate	Monitoring, alerting, patch management	SIEM

NEN-7510:2024-2 relevante controls

Hoofdstuk 5

- **5.23** Informatiebeveiliging voor het gebruik van clouddiensten
- **5.30** ICT-gereedheid voor bedrijf continuïteit
 - BCM, disaster recovery; hoe weer live gaan als productie 'boem' gedaan heeft.

Hoofdstuk 8

- **8.4 & 8.32** - Toegangsbeveiliging op broncode & wijzigingsbeheer
- **8.8** – Beheer van technische kwetsbaarheden
- **8.9** – Configuratiebeheer
- **8.16** – Monitoren van activiteiten
- **8.25 & 8.29** – Beveiligen & testen tijdens de ontwikkelcyclus
- **8.28** – Veilig programmeren
- **8.31** – Scheiding van ontwikkel- test- en productieomgeving
- **8.33** - Testgegevens

*details in volgende slides!

8.4 & 8.32 | Broncode & change mgmt

“Wie mag wat aanraken — en hoe bewijsbaar is dat?”

- **Eis:**

- Toegang tot broncode is beperkt. Alle wijzigingen zijn traceerbaar naar een natuurlijk persoon.

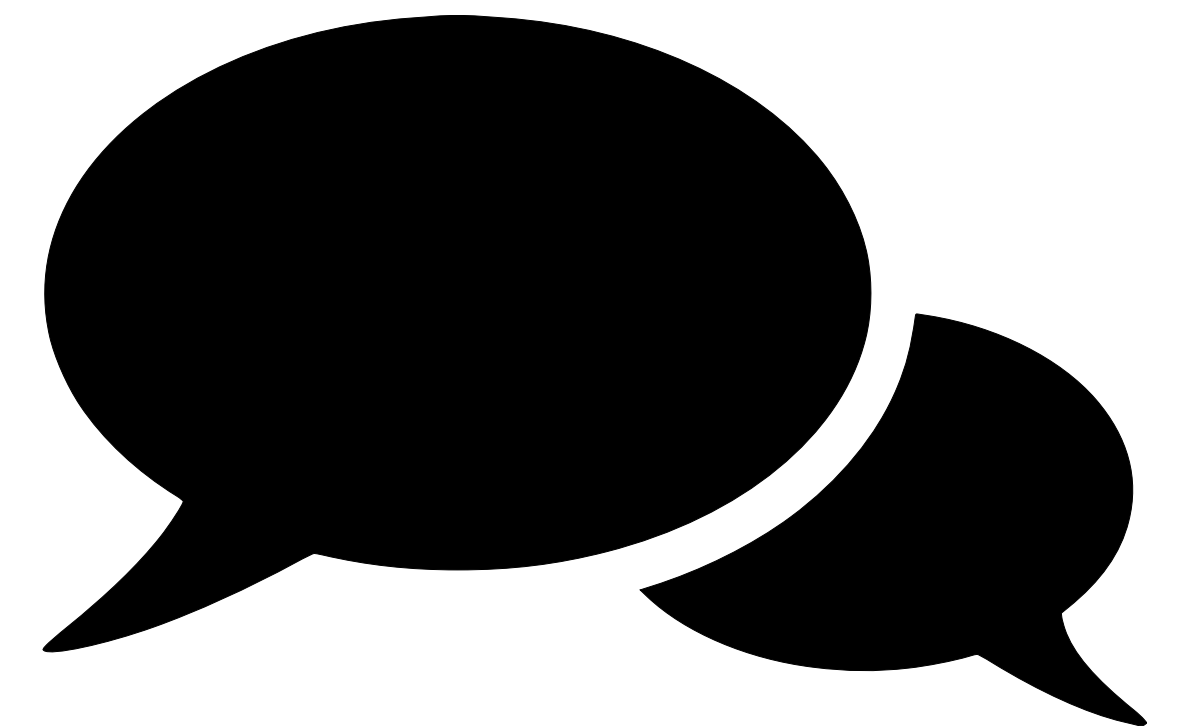
- **Maatregelen:**

- MFA verplicht voor alle GitHub-accounts (ook admins)
- RBAC: niet elke ontwikkelaar heeft admin of schrijf rechten op elk repo
- Protected branches (master/release): alleen via Pull Request
- Minimaal 1 code review verplicht
- Alle CI-checks moeten slagen vóór merge
- Force-push en directe deletes geblokkeerd — ook voor admins
- Branch ruleset: geldt voor iedereen zonder uitzondering
- Signed commits (optioneel maar aanbevolen): cryptografisch bewijs van auteur
 - PGP
- Audit log in GitHub: wie heeft wanneer wat gewijzigd

8.4 & 8.32 | Broncode & change mgmt

En deze?

- AI-agents
 - QOTD: "RCE As A Service"
- MCP-integraties (GitHub & consorten)
- machine-to-machine koppelingen



Vragen:

- Hoe zorg je dat een AI-agent ook valt onder least privilege?
- Hoe verifieer je dat een door AI gegenereerde commit veilig is?
- Wie is verantwoordelijk als een AI-agent een kwetsbaarheid introduceert?

8.8 | Beheer van techn. vulns

“Weet wat er in je software zit — en houd het bij”

- **Eis:**
 - Kwetsbaarheden worden tijdig geïdentificeerd, beoordeeld en verholpen.

Tool/Maatregel	Wat doet het?
SAST	Analyseert broncode op bekende kwetsbaarheden <i>vóór</i> compilatie — CodeQL, Semgrep
SCA	Controleert third-party dependencies op CVE's — Dependabot, Snyk, OWASP DC
SBOM	Inventaris van alle componenten + versies — CycloneDX of SPDX formaat
Patch mgmt	Harde deadlines op basis van CVSS-score én kritiekheid van het systeem

8.8 | Beheer van techn. vulns

“Weet wat er in je software zit — en houd het bij”

- **Patch-prioriteit (richtlijn):**

- CVSS 9–10 (Critical): ≤ 24 uur
- CVSS 7–8.9 (High): ≤ 1 week
- CVSS 4–6.9 (Medium): volgende sprint
- CVSS < 4 (Low): geplande release

- **CVSS?**

- Common Vulnerability Scoring System
- Standaard framework om de ernst van beveiligingsproblemen in te schatten op een schaal van 0 tot 10.
- De score wordt bepaald op basis van een aantal factoren, o.a.:
 - hoe het probleem kan worden misbruikt (aanvalsvector)
 - hoe moeilijk dat is
 - of toegang nodig is
 - wat de gevolgen zijn voor beveiliging, gegevensintegriteit en beschikbaarheid (CIA)

8.9 | Configuratiebeheer

“De pipeline zelf is ook software — moet dus beveiligd worden”

- **Eis:**

- Configuraties worden beheerd, gedocumenteerd en beschermd tegen ongeautoriseerde wijziging.

- **Maatregelen**

- Pipeline-as-code: GitHub Actions workflows staan in de repo (.github/workflows/)
 - Versiebeheerd, reviewable, auditable
- Quality gates
 - pipeline blokkeert bij falen van checks (Build → Unit tests → SAST → SCA → (DAST) → review → deploy)
- Secrets management
 - nooit hardcoded in code of workflow-bestanden
 - Gebruik GitHub Secrets / Environment Secrets (niet direct leesbaar, wel 'auditable')
- Immutable artifacts
 - een gebouwde artifact (JAR, container image) wordt niet achteraf gewijzigd;
 - een fix = nieuwe build

8.25 | Beveiligen tijdens ontwikkelen

“Security by design: van eerste commit tot productie”

- Secure coding standards gedefinieerd en afgedwongen
 - zie 8.28
- Threat modelling in de designfase
 - b.v. STRIDE of PASTA
- Omgevingen zijn gescheiden
 - zie 8.31
- AuthN & AuthZ als expliciete ontwerpeis
 - NIET "we voegen het later toe"
- Safe defaults
 - systeem is standaard veilig geconfigureerd

8.29 | Testen(security focus)

“Security by design: van eerste commit tot productie”

- SAST in de CI-pipeline
 - elke commit
- Dependency Review bij elke PR
 - blokkeer nieuwe kwetsbare deps
- DAST in de acceptatieomgeving
 - B.v. OWASP ZAP
- Beveiligingstest-resultaten worden opgeslagen als pipeline-artifact
 - == bewijs
- Penetratietest bij releases
 - Bij voorkeur door specialisten

8.28 Safe coding

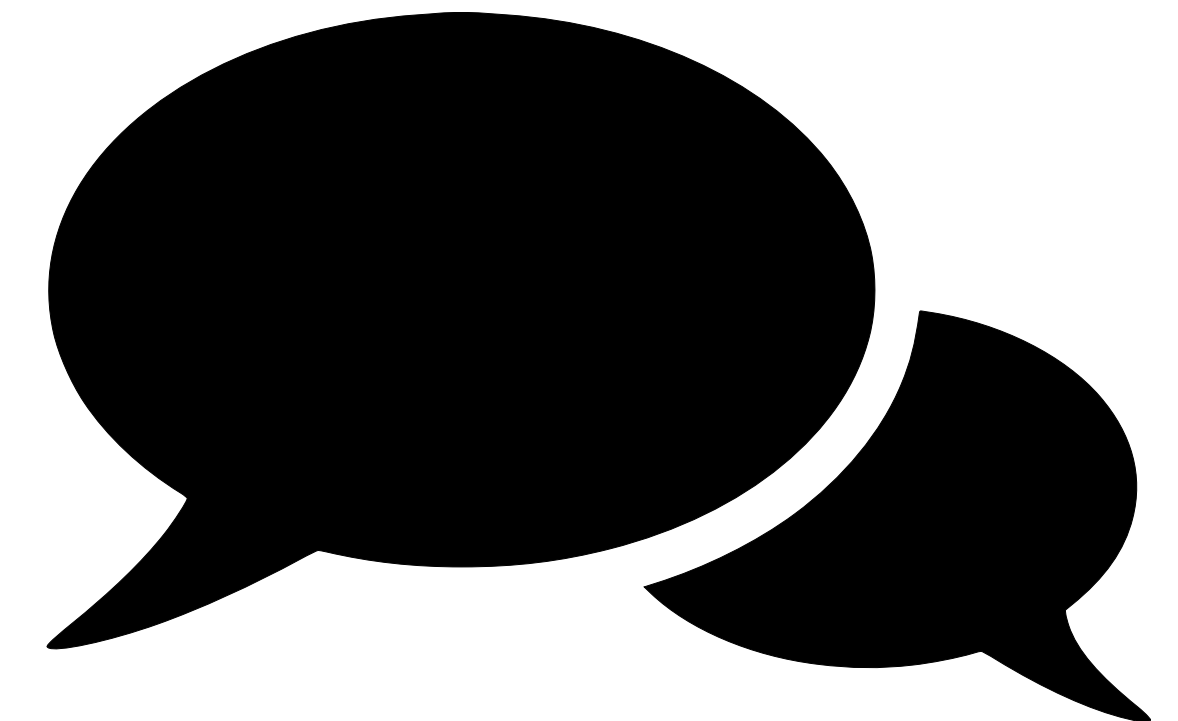
- **Eis:**
 - Er gelden vastgestelde richtlijnen voor veilig coderen **en** deze worden gecontroleerd.
- **Maatregelen**
 - **Coding standards** gedocumenteerd
 - b.v. OWASP Secure Coding Practices
 - **Linters** afdwingen stijl én security-patronen
 - Java/Maven: SpotBugs, PMD, Checkstyle
 - In GitHub Actions: falen bij violations
 - **Secrets-beheer** in code:
 - Nooit API-keys, wachtwoorden of tokens in code of .env bestanden in de repo
 - GitHub Secret Scanning detecteert per ongeluk ge-committe secrets
 - Pre-commit hooks als extra vangnet (b.v. `'detect-secrets'`)
 - **Dependency pinning:** gebruik exacte versienummers en hashes
 - Niet `'latest'`.

Sidenote: wanneer dependencies bijwerken

- **Was:**
 - Z.s.m.
- **Nu**
 - ‘It Depends’ (get used to it)
 - Veel supply chain aanvallen
 - Detectievenster uren tot dagen
 - Best practice neigt nu naar
 - **‘wacht een paar dagen met bijwerken tenzij...’**
 - (buzzword of the week: ‘dependency cooldown’)
 - Staat op gespannen voet met tijd tussen 0-day bekend → mass exploitation
 - Nieuws: TeamPCP aanvallen op o.a. Trivy en Github
 - <https://wiz.ondemand.goldcast.io/on-demand/48163bfd-951d-4d21-b1fe-f07f2fd1e000>



Wat als een secret wel in de git-history terechtkomt?



<https://arstechnica.com/information-technology/2026/05/in-stunning-display-of-stupid-secret-cisa-credentials-found-in-public-github-repo/>

CISA: Cybersecurity and Infrastructure Security Agency

Was tot twee jaar geleden 'Top of the line', nu 'GeTrumpifiseerd'

https://en.wikipedia.org/wiki/Cybersecurity_and_Infrastructure_Security_Agency

8.31 | Scheiding van omgevingen (OTAP)

Omgeving	Doel	Data
Ontwikkel	Lokaal / feature branches	Gesynthetiseerde data
Test	Geautomatiseerde tests	Gegenereerde / geanonimiseerde data
Acceptatie	Functionele validatie + UAT	Gepseudonimiseerde data
Productie	Live systeem	Echte patiëntdata

Harde regels

- Productiedata gaat **nooit** naar een lagere omgeving
- Systemen uit verschillende omgevingen communiceren **nooit** met elkaar
- Gebruik stubs/mocks voor third-party services in test/acceptatie
- Gescheiden secrets per omgeving (GitHub Environments)

8.33 | Testdata

- Realistische tests vereisen realistische data — **maar niet echte data**
- Pseudonimisering is technisch moeilijk goed te krijgen
 - correlatie van velden maakt de-anonimisering mogelijk
 - GenAI is 'best wel' goed in correleren...
- Gebruik testdata-generatoren
 - b.v. Synthea voor medische data
- Gevoelige data z.s.m. verwijderen uit testomgevingen na gebruik

8.16 | Monitoring van de pipelines

“De pipeline is ook een aanvalsoppervlak”

- **Eis:**
 - Activiteiten worden gelogd en gecontroleerd op afwijkingen.

Wat monitoren in een CI/CD Context?

- **Failed pipeline runs**
 - patroon van mislukte builds kan sabotage of fout signaleren
- **Gebruik van secrets**
 - wie heeft wanneer een secret opgevraagd?
- **Wijzigingen in workflow-bestanden**
 - (.github/workflows/)
 - direct een alert
- **Deployment events**
 - wie heeft wat naar welke omgeving deployed en wanneer?
- **Integriteit van artifacts**
 - hash van build-output opslaan en verifiëren bij deploy
- **Beheeraccounts**
 - alert bij gebruik van admin-rechten buiten kantoortijden

8.16 | Monitoring van de pipelines

“De pipeline is ook een aanvalsoppervlak”

- **Eis:**

- Activiteiten worden gelogd en gecontroleerd op afwijkingen.

Tooling?

- GitHub Security tab + Dependabot alerts
- GitHub Audit Log
 - exporteerbaar naar SIEM
- Notifications / webhooks
 - naar Slack/Teams voor kritieke events

5.23 | Beveiliging cloud services

“GitHub is een clouddienst”

- Vastleggen welke clouddiensten gebruikt worden
 - en met welk doel!
- Toegang tot GitHub beheerd via organisatie-accounts
 - **geen** persoonlijke accounts
- SSO / SAML koppeling met organisatie-IdP
 - b.v. Azure AD (Entra) (voor dit project optioneel)
- Data-retentiebeleid:
 - hoe lang pipeline-logs en artifacts bewaren?
- **Alle bovenstaande punten vastleggen!**

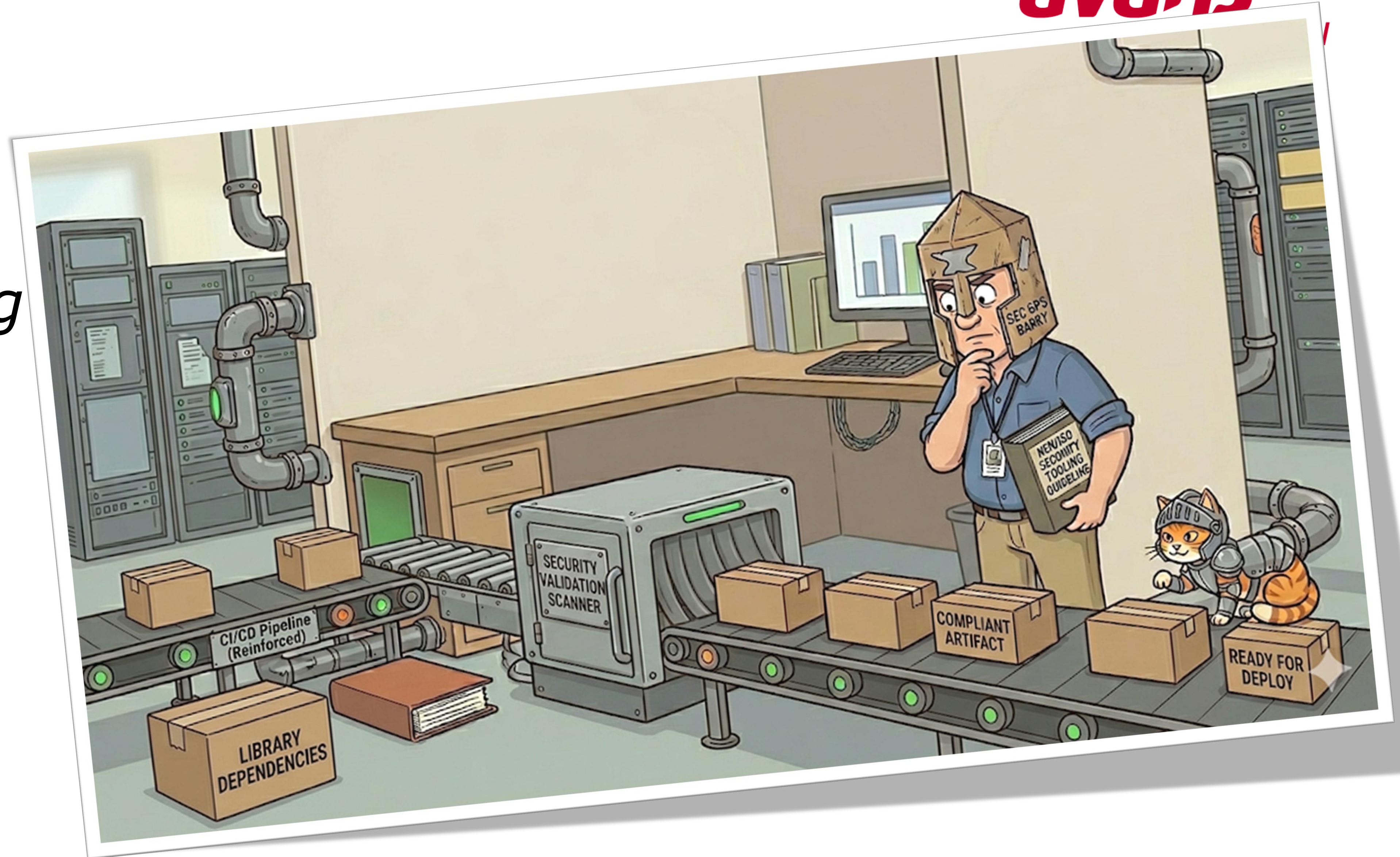
5.30 | BCM

"Business Continuity Management"

(voor dit project even minder relevant)

- Wat als GitHub onbereikbaar is?
- Backup-strategie voor de repository (mirror naar eigen infra)
- Disaster recovery procedure: hoe ga je weer live na een incident?
- Wie mag een rollback uitvoeren?
- Hoe snel moet het systeem beschikbaar zijn?
 - RTO: hoe snel (Return Time Objective)
 - RPO: wat kunnen we missen (Return Point Objective)
- CI/CD-pipeline zelf is onderdeel van het continuïteitsplan

Tooling



GitHub Security tooling

Feature	Wat het doet	NEN-7510
Branch Protection / Rulesets	Afdwingen van PR + reviews + checks	8.4, 8.32
Secret Scanning	Detecteert ge-committe secrets	8.28
CodeQL	SAST — vindt security-bugs in code	8.8, 8.29
Dependabot Alerts	CVE-melding bij kwetsbare deps	8.8
Dependabot Security Updates	Automatische PR's voor patches	8.8
Dependency Review Action	Blokkeert PR's met nieuwe kwetsbare deps	8.8, 8.29
GitHub Environments	OTAP met approval-gates en gescheiden secrets	8.31
Audit Log	Wie deed wat wanneer in de org	8.4, 8.16
SBOM export	Dependency-inventaris (CycloneDX/SPDX)	8.8

GitHub tips

Denk na over de inrichting

- Maak een organisatie aan
- Maak gebruik van projecten
- Richt issues / tickets / backlog goed in
- Richt
- Doe dit NU, dan heb je er het meeste profijt van. Docs o.a.:
 - <https://docs.github.com/en/organizations>
 - <https://docs.github.com/en/enterprise-cloud@latest/code-security/getting-started/github-security-features>
 - <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>
- Best wel veel werk. Verdeel het dus binnen je project (review de handelingen ook!)



Note: de 'action' configuraties op de volgende zijn ter illustratie en niet volledig!

OpenMRS: Java / Maven stack

- Modules worden gebouwd als .omod-bestanden (OSGi bundles)
- SAST voor Java / Maven

.github/workflows/sast.yml (fragment)

- name: Initialize CodeQL
uses: github/codeql-action/init@v3
with:
 languages: java
- name: Build with Maven
run: mvn -B package --no-transfer-progress
- name: Perform CodeQL Analysis
uses: github/codeql-action/analyze@v3

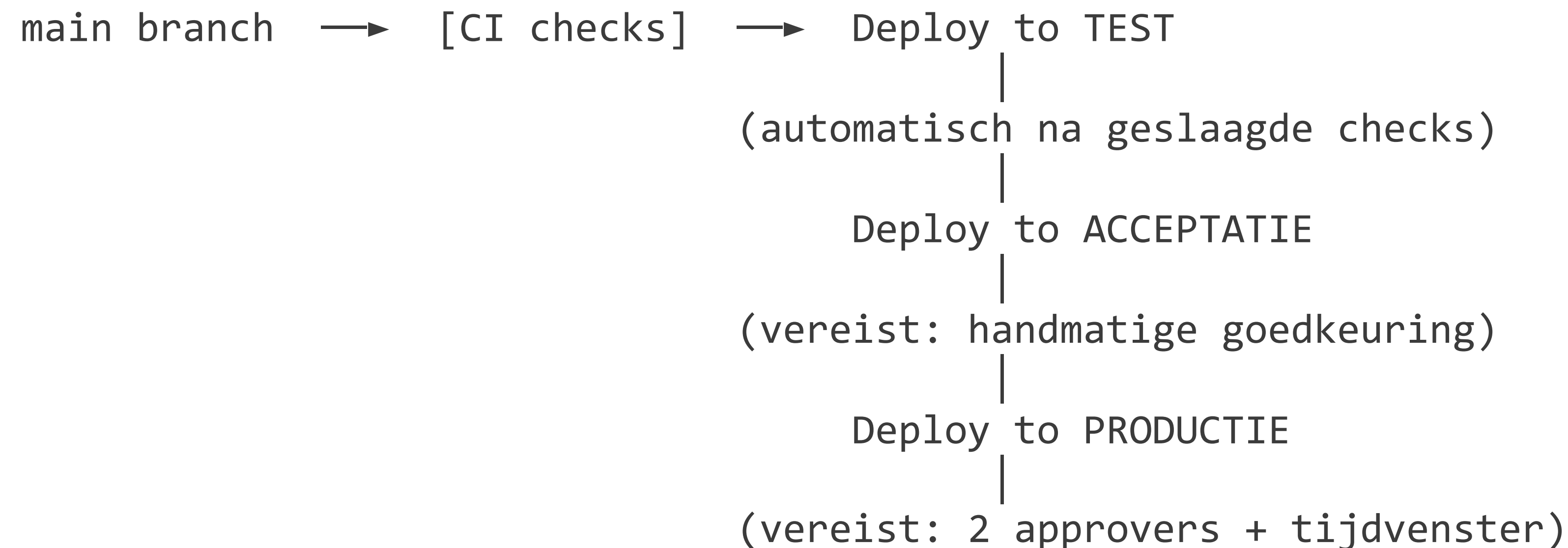
OpenMRS: Java / Maven stack

- SCA voor Maven
 - `name: Dependency Review`
`uses: actions/dependency-review-action@v4`
`with:`
`fail-on-severity: high`

OpenMRS: Java / Maven stack

- SBOM genereren
 - `name: Generate SBOM`
`uses: anchore/sbom-action@v0`
`with:`
 - `format: cyclonedx-json`
 - `output-file: sbom.cyclonedx.json`
 - `name: Upload SBOM as artifact`
`uses: actions/upload-artifact@v4`
`with:`
 - `name: sbom`
 - `path: sbom.cyclonedx.json`

OTAP in GitHub (environments)



- Per omgeving:
 - Eigen secrets (database-wachtwoorden, API-keys — nooit gedeeld tussen omgevingen)
 - Protection rules: wie mag goedkeuren, tijdvenster (bijv. alleen kantooruren voor productie)
 - Audit trail: elke deployment gelogd met actor, tijdstip en versie

GitHub → NEN-7510

GitHub-maatregel	NEN-7510 control	Bewijs
MFA verplicht voor alle accounts	8.4, 8.5	Org-instelling screenshot
Branch protection + required reviews	8.4, 8.32	Ruleset-configuratie
CodeQL in CI	8.8, 8.29	Pipeline-run rapport
Dependabot alerts + updates	8.8	Alert-overzicht
Dependency Review Action op PR's	8.8, 8.29	PR-check log
Secret Scanning actief	8.28	Security tab
SBOM gegenereerd als artifact	8.8	Artifact in GitHub Actions
Environments met protection rules	8.31	Environment-configuratie
Secrets per environment gescheiden	8.31, 8.33	Environment secrets screenshot
GitHub Audit Log	8.16, 8.4	Exporteerbaar log
README met beleid en procedures	8.9, 8.25	README.md in repo

Opdracht 1 | OTAP inrichten

Met de projectgroep

Doel: Een aantoonbaar geharde CI/CD-omgeving voor het OpenMRS-project.

Stappen:

- Maak minimaal **test** en **productie** omgevingen aan
 - Gescheiden configuratie, gescheiden secrets
 - Als docker gebruikt wordt kan dit met docker-compose bestanden
- Configureer **GitHub Environments** (minimaal test + productie)
 - Protection rules + approval-gates
- Richt de NEN-7510 controls in (zie checklist volgende slide)
- Schrijf een **README.md** die beschrijft:
 - Hoe de omgevingen zijn ingericht
 - Hoe voorkomen wordt dat testdata in productie terechtkomt
 - Hoe een nieuwe ontwikkelaar onboarded

Een nieuwe ontwikkelaar moet met alleen de README aan de slag kunnen.

Opdracht 1 | Checklist

Minimale eisen voor een aantoonbaar compliant pipeline

- [] Branch protection actief op main — alleen via PR, reviews verplicht
- [] Alle CI-checks slagen vóór merge (build, test, SAST)
- [] **CodeQL** of gelijkwaardige SAST actief (bijv. Snyk)
- [] **Secret Scanning** actief
- [] **Dependabot** alerts + security updates actief
- [] **Dependency Review Action** gekoppeld aan PR's
- [] **SBOM** wordt gegenereerd (CycloneDX of SPDX formaat) en geanalyseerd (**SCA**)
- [] **GitHub Environments** gedefinieerd met protection rules
- [] Secrets zijn gescheiden per environment
- [] Pipeline-artifacts (rapporten, SBOM) worden bewaard
- [] **README.md** beschrijft beleid en procedure (= mini-ISMS)

Opdracht 2 | Compliance verslag

Met de projectgroep

Doel: Aantonen dat de pipeline compliant is.

(opdracht 1 hoeft nog niet 100% gereed te zijn):

Schrijf een compliance verslag met:

- Per relevante **NEN-7510:2024-2 control**:
 - Wat de control vereist (korte beschrijving)
 - Hoe de pipeline hieraan voldoet (met verwijzing naar concrete instelling/bestand/screenshot)
 - Welk restrisico er nog is (wat is nog niet geïmplementeerd en waarom)
- Structuur:
 - tabel of genummerde secties per control
- Gebruik de controls uit slide 9 als basis
- **Oplevering:**
 - Markdown of Word document, ingeleverd via de repo.

Recap

- Een secure CI/CD-pipeline is in de healthcare-context **wettelijk** vereist
 - NEN-7510, CBW/NIS2
- DevSecOps == Shift Left
 - security zit in elke stap, niet alleen aan het einde
- De NEN-7510:2024-2
 - controls 8.4 t/m 8.33 zijn direct te mappen op concrete GitHub-features
- OTAP-omgevingen
 - gescheiden secrets en approval-gates zijn de technische invulling van 8.31
- Een pipeline is ook bewijs
 - artifacts, logs en audit trails zijn de compliance-documentatie
- Compliant by design
 - regelgeving is een ontwerpeis, geen afvinklijstje

Volgende keer

- Healthcare Risk Assessment
- Zorg dat je bij blijft
- **Nogmaals: bulk commit in week 8 is NIET compliant werken...**



TOT SLOT.

Vragen?